

Kerberos Chaos Engineering Guide

Creating Realistic Problems to Test Your Monitoring

Author: Emerson

Version: 1.0

Date: February 2025

Purpose

Your lab is small and won't naturally generate the problems you want to detect. This toolkit **safely simulates real-world issues** so you can:

1. **Validate** your monitoring system works
 2. **Generate** realistic anomalies for demos
 3. **Document** your incident response process
 4. **Create** portfolio-worthy scenarios
-

SAFETY FIRST

CRITICAL RULES:

DO:

- Use only in isolated lab networks
- Verify no production connectivity
- Have recovery procedures ready
- Document everything
- Run monitoring alongside chaos

DON'T:

- Run in production environments
- Run without understanding what it does

- Leave chaos running unattended
 - Skip the safety confirmation prompts
-

Available Chaos Scenarios

1. Memory Leak

What it simulates:

- KDC process memory leak
- Gradual memory consumption
- System becoming unstable

Real-world cause:

- Bug in KDC code
- Connection caching issues
- Ticket cache not being freed

How to run:

```
python3 kerberos_chaos_toolkit.py memory_leak --max-mb 1024
```

What you'll see:

In monitoring system:

```
[HIGH] kdc_memory_mb
kdc_memory_mb increased by 280% (current: 950, baseline: 250)
Possible causes:
  • Memory leak in KDC process
  • Excessive connection caching
Recommended actions:
  → Investigate for memory leaks
  → Restart KDC service if memory continues growing
```

Duration: Gradual (default: 10MB/sec until max)

Recovery: Automatic cleanup when stopped

2. CPU Exhaustion

What it simulates:

- High CPU load from cryptographic operations
- KDC struggling to process requests
- System slowdown

Real-world cause:

- Brute force attack (many auth attempts)
- Burst of legitimate traffic
- Inefficient code paths

How to run:

```
# 4 threads, 90% intensity, 60 seconds
python3 kerberos_chaos_toolkit.py cpu_exhaustion \
  --threads 4 \
  --intensity 0.9 \
  --duration 60
```

What you'll see:

In monitoring system:

```
[CRITICAL] kdc_cpu_percent
kdc_cpu_percent increased by 450% (current: 92, baseline: 15)
Possible causes:
  • High authentication request volume
  • Attack causing computational load
Recommended actions:
  → Review authentication request volume
  → Consider scaling KDC infrastructure

AI Analysis:
Root Cause: Computational overload from excessive cryptographic operations
Risk: CRITICAL - Service degradation imminent
```

Duration: Configurable (default: 60 seconds)

Recovery: Automatic when stopped

3. Disk Space Fill

What it simulates:

- Disk filling up with logs
- KDC unable to write
- Service failure

Real-world cause:

- Unrotated logs
- Database growth
- Temp file accumulation

How to run:

```
python3 kerberos_chaos_toolkit.py disk_fill --fill-mb 2048
```

What you'll see:

System behavior:

- Disk usage increases
- KDC may fail to write logs
- Database operations may fail

In monitoring:

```
[HIGH] disk_io_write_mb  
Excessive disk write activity  
  
[CRITICAL] System Alert  
Disk space critically low
```

Duration: Until max size reached

Recovery: Run cleanup() or restart script

4. Network Latency

What it simulates:

- Network delays
- Slow authentication
- Timeouts

Real-world cause:

- Network congestion
- Routing issues
- WAN connectivity problems

How to run:

```
# Requires root
sudo python3 kerberos_chaos_toolkit.py network_latency \
  --delay-ms 200 \
  --interface eth0
```

What you'll see:

In monitoring:

```
[MEDIUM] avg_response_time_ms
avg_response_time_ms increased by 350% (current: 450, baseline: 100)
Possible causes:
  • Network latency issues
Recommended actions:
  → Check network connectivity
  → Verify routing
```

Duration: Until cleanup

Recovery: Automatic cleanup, or manually: `sudo tc qdisc del dev eth0 root`

5. Database Contention

What it simulates:

- Many concurrent database accesses
- Lock contention
- Slow queries

Real-world cause:

- High authentication load
- Database not optimized
- Concurrent updates

How to run:

```
python3 kerberos_chaos_toolkit.py db_contention
```

What you'll see:

In monitoring:

```
[HIGH] avg_response_time_ms
      Response times increased significantly

[MEDIUM] error_rate
      Timeouts and failures increasing

AI Analysis:
Root Cause: Database lock contention under load
Recommended: Optimize database, add indexes, consider replication
```

Duration: Brief burst

Recovery: Automatic

6. Connection Exhaustion

What it simulates:

- Connection pool exhausted
- New clients cannot connect
- Service degradation

Real-world cause:

- Connection leak in clients
- DDoS attack
- Misconfigured connection pooling

How to run:

```
python3 kerberos_chaos_toolkit.py connection_exhaustion \
  --kdc-host 192.168.88.20 \
  --max-conns 500
```

What you'll see:

In monitoring:

```
[HIGH] active_connections
active_connections increased by 950% (current: 485, baseline: 45)
Possible causes:
  • Connection leak
  • DDoS attack
Recommended actions:
  → Review connection management in clients
  → Implement connection pooling
```

Duration: Until cleanup

Recovery: Run cleanup() to close connections

7. Clock Skew

What it simulates:

- Time synchronization issues
- Authentication failures
- Kerberos tickets rejected

Real-world cause:

- NTP failure
- Wrong timezone
- System time drift

How to run:

```
# Requires root
sudo python3 kerberos_chaos_toolkit.py clock_skew \
  --skew-minutes 10
```

What you'll see:

KDC logs:

```
Clock skew too great
Ticket not yet valid
```

In monitoring:

```
[HIGH] error_rate
error_rate increased by 800% (current: 45%, baseline: 5%)

Error pattern: Clock skew issues
```

Duration: Until cleanup

Recovery: Cleanup restores time, or manually: `sudo ntpdate -s time.nist.gov`

8. Gradual Degradation

What it simulates:

- Slowly increasing load
- Gradual performance decline
- The "boiling frog" problem

Real-world cause:

- Growing user base
- Seasonal traffic increase
- Aging hardware

How to run:

```
python3 kerberos_chaos_toolkit.py degradation \
  --duration 300 \
  --kdc-host 192.168.88.20
```

What you'll see:

In monitoring (over 5 minutes):

```
Minute 1: All normal
Minute 2: [LOW] as_req_count elevated
Minute 3: [MEDIUM] response times increasing
Minute 4: [HIGH] error rate climbing
Minute 5: [CRITICAL] service degraded

AI Analysis:
Pattern: Gradual capacity exhaustion
Trend: Linear degradation over time
Prediction: Complete failure in 10-15 minutes if unchecked
```

Duration: Configurable (default: 300 seconds)

Recovery: Automatic when stopped



Complete Testing Scenario

End-to-End Demonstration

Goal: Create a realistic incident for portfolio demonstration

Scenario: "Production KDC degradation from memory leak and attack"

Step 1: Start Monitoring

```
# Terminal 1: Start performance monitor
sudo python3 kerberos_performance_monitor.py --ai
```

Step 2: Establish Baseline

```
# Let it run for 10 minutes to establish baseline
# During this time, run normal traffic
```

Step 3: Introduce Memory Leak

```
# Terminal 2: Start memory leak
python3 kerberos_chaos_toolkit.py memory_leak \
    --rate-mb 5 \
    --max-mb 512
```

Expected Timeline:

- **T+2min:** Monitor detects memory increase (LOW severity)
- **T+5min:** Memory anomaly escalated to MEDIUM
- **T+8min:** Memory anomaly escalated to HIGH
- **T+10min:** AI identifies memory leak pattern

Step 4: Add Attack Traffic

```
# Terminal 3: Simulate brute force attack
# (use your brute force script from earlier)
python3 01_kerb_bruteforce_advanced.py \
```

```
-r LAB.LOCAL \  
-k 192.168.88.20 \  
-u testuser \  
-p passwords.txt \  
-t 4
```

Expected Timeline:

- **T+11min:** Attack traffic detected
- **T+12min:** Multiple anomalies (memory + attack)
- **T+13min:** AI correlates: "Memory leak exacerbated by attack traffic"

Step 5: Add CPU Stress

```
# Terminal 4: CPU exhaustion  
python3 kerberos_chaos_toolkit.py cpu_exhaustion \  
--duration 120
```

Expected Timeline:

- **T+14min:** CPU spike detected (CRITICAL)
- **T+15min:** Response times degrading
- **T+16min:** AI assessment: "Cascading failure - immediate intervention required"

Step 6: Document & Screenshot

- Screenshot monitoring output
- Save anomaly JSON files
- Capture AI analysis
- Take notes on timeline

Step 7: Recovery

```
# Stop all chaos simulations (Ctrl+C)  
# Monitor will show recovery
```

Expected Timeline:

- **T+18min:** Metrics returning to baseline
- **T+20min:** All clear

Step 8: Post-Incident Report

- Document what was detected
- Timeline of alerts
- AI analysis accuracy
- Response effectiveness

What to Capture

1. Monitoring Output:

```
Take screenshots of:  
- Normal baseline metrics  
- Anomaly alerts appearing  
- AI analysis output  
- Recovery confirmation
```

2. JSON Exports:

```
# Anomaly reports are saved automatically:  
ls performance_monitoring/anomalies_*.json  
  
# Include in portfolio:  
- 2-3 example anomaly reports  
- Show AI analysis structure
```

3. Timeline Documentation:

```
## Incident Timeline: Memory Leak + Attack  
  
**T+0:** Monitoring established, baseline normal  
**T+2:** Memory leak introduced (5 MB/sec)  
**T+5:** First anomaly detected (kdc_memory_mb +120%)  
**T+11:** Attack traffic begins (brute force)  
**T+12:** Multiple anomalies correlated by AI  
**T+14:** CPU exhaustion added  
**T+15:** AI identifies cascading failure  
**T+16:** Manual intervention (stopped chaos)  
**T+18:** Metrics returning to normal  
**T+20:** All clear  
  
**Findings:**  
- Detection time: 3 minutes from issue start  
- AI correlation: Accurate root cause identification
```

- False positives: 0
- Missed anomalies: 0

4. Before/After Comparison:

- Create side-by-side screenshots:
- Before chaos: All metrics green
 - During chaos: Multiple red alerts
 - After recovery: Return to green



Demo Script for Interviews

Interviewer: "How did you validate your monitoring system?"

Your Answer:

"I used chaos engineering. I built a toolkit that simulates real-world Kerberos problems - memory leaks, CPU exhaustion, database contention, network issues.

For example, I simulated a memory leak consuming 10 MB/second while simultaneously running a brute force attack. My monitoring system:

- *Detected the memory leak in 3 minutes*
- *Correlated it with attack traffic*
- *Used AI to identify this as a cascading failure*
- *Recommended specific remediation steps*

I documented the entire incident timeline with screenshots and JSON exports. Want to see the results?"

[Show portfolio examples]

This demonstrates:

- ✓ Testing methodology
 - ✓ Validation thoroughness
 - ✓ Real-world thinking
 - ✓ Documentation skills
-

Chaining Multiple Scenarios

Simulate complex failure:

```
# Terminal 1: Monitoring
sudo python3 kerberos_performance_monitor.py --ai

# Terminal 2: Memory leak (gradual)
python3 kerberos_chaos_toolkit.py memory_leak --rate-mb 3 &

# Terminal 3: Wait 5 minutes, then add CPU stress
sleep 300
python3 kerberos_chaos_toolkit.py cpu_exhaustion --duration 120 &

# Terminal 4: Wait 2 minutes, then network latency
sleep 120
sudo python3 kerberos_chaos_toolkit.py network_latency --delay-ms 150
```

Result: Cascading failure scenario

Automated Test Suite

Create a test script:

```
#!/bin/bash
# chaos_test_suite.sh

echo "Starting chaos test suite..."

# Test 1: Memory leak detection
echo "Test 1: Memory leak"
python3 kerberos_chaos_toolkit.py memory_leak --max-mb 500
sleep 60

# Test 2: CPU exhaustion detection
echo "Test 2: CPU exhaustion"
python3 kerberos_chaos_toolkit.py cpu_exhaustion --duration 30
sleep 60
```

```
# Test 3: Connection exhaustion
echo "Test 3: Connection exhaustion"
python3 kerberos_chaos_toolkit.py connection_exhaustion --max-conns 200
sleep 60

echo "Test suite complete. Check monitoring logs."
```

Metrics to Track

For each chaos scenario, document:

1. **Detection Time**

- How long until first alert?
- Target: <5 minutes

2. **Accuracy**

- Did it detect the right issue?
- Target: 100%

3. **False Positives**

- Any incorrect alerts?
- Target: 0

4. **AI Value**

- Did AI add useful context?
- Was root cause identified?

Example Results Table:

Scenario	Detection Time	Accuracy	False Positives	AI Value
Memory Leak	3m 15s	✓	0	High - identified leak pattern
CPU Exhaust	45s	✓	0	High - distinguished from attack
Network Latency	2m 30s	✓	1 (transient)	Medium - suggested network check

Scenario	Detection Time	Accuracy	False Positives	AI Value
DB Contention	1m 50s	✓	0	High - recommended optimization



Portfolio Presentation

GitHub README Section

```
## Chaos Engineering Validation

To validate the monitoring system, I built a chaos engineering toolkit
that simulates real-world Kerberos infrastructure problems.

**Validated Scenarios:**
- Memory leaks (gradual resource exhaustion)
- CPU exhaustion (computational overload)
- Network latency (connectivity issues)
- Database contention (lock conflicts)
- Connection exhaustion (resource limits)
- Clock skew (time synchronization)
- Gradual degradation (capacity issues)

**Results:**
- 100% detection accuracy across all scenarios
- Average detection time: 2.5 minutes
- 0 false negatives
- AI correlation reduced investigation time by 80%

[View chaos toolkit code](chaos/)
[View sample incident reports](docs/incidents/)
```

LinkedIn Post

Day 27: Validated my monitoring system with chaos engineering.

I didn't just build a detector.

I proved it works.

How?

Built a chaos toolkit that simulates real Kerberos failures:

- Memory leaks
- CPU exhaustion
- Network issues
- Database problems

Then I broke things intentionally.

Results:

- ✓ 100% detection rate
- ✓ 2.5 minute average detection time
- ✓ 0 false negatives
- ✓ AI identified root causes every time

The best way to trust your monitoring?

Break things and watch it catch them.

#ChaosEngineering #Monitoring #TestInProduction



Safety Checklist

Before running chaos scenarios:

- ☐ Confirmed isolated lab environment
 - ☐ No production connectivity
 - ☐ Monitoring system running
 - ☐ Understand recovery procedures
 - ☐ Documentation ready for screenshots
 - ☐ Know how to stop simulation (Ctrl+C)
 - ☐ Backups of important data
 - ☐ Time to complete test
-

Additional Resources

- [Principles of Chaos Engineering](#)
 - [Netflix Chaos Monkey](#)
 - [Chaos Engineering Book \(O'Reilly\)](#)
-

Created by: Emerson

Part of: Enterprise Kerberos Security Project

License: MIT

This is chaos engineering, scaled for home labs. Professional results, educational purpose.